

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 8

з дисципліни «Алгоритмізація процесів управління»

Тема: "Реалізація алгоритмів сортування та робота з файлами на мові C ++"

XAI.301.G3. 319a. 16 ЛР

Виконав студент гр. _____ 319а _____

_____ Карина САФОНОВА _____
(підпис, дата) (П.І.Б.)

Перевірив

_____ асис. Євгеній ПЯВКА (підпис, дата)
(П.І.Б.)

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_n.txt». Array97 Дан цілочисельний масив розміру N. Видалити з масиву всі однакові елементи, залишивши їх останні входження.

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_n.txt» з елементами вихідного двовимірного масиву (n - номер варіанта). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл. Matrix23. Дана матриця розміру $M \times N$. У кожному рядку матриці знайти мінімальний елемент.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

14	Бульбашкова	Зменшення	Дійсний
----	-------------	-----------	---------

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

```
main.cpp //основна функція і три функції для 3х завдань
array_utils.h //заголовки функцій для роботи з
//одновимірними масивами (завдання1,3)
array_utils.cpp //визначення функцій для роботи з
//одновимірними масивами (завдання1,3)
matrix_utils.h //заголовки функцій для роботи з
```

```
//двовимірними масивами (завдання2)
matrix_utils.cpp //визначення функцій для роботи з
//двовимірними масивами (завдання2)
```

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

- 1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»
 - 2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»
 - 3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»
- Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі

Array97 Дан цілочисельний масив розміру N. Видалити з масиву всі однакові елементи, залишивши їх останні входження.

Вхідні дані (ім'я, опис, тип, обмеження):

N – розмір масиву, тип int, обмеження: $2 \leq N \leq 50$;

arr[N] – цілочисельний масив, тип int.

Вихідні дані (ім'я, опис, тип):

`arr[N]` – змінений масив без повторюваних елементів (залишені тільки останні входження), тип `int`; `N` – новий розмір масиву після видалення повторів, тип `int`, обмеження: $1 \leq N \leq 50$.

Алгоритм вирішення показано нижче

1. Відкрити файл для читання.
2. Зчитати кількість елементів `n`.
3. Зчитати елементи масиву.
4. Пройти масив з кінця, запам'ятовуючи, які елементи вже зустрічалися.
5. Сформувати новий масив лише з останніх входжень кожного унікального числа.
6. Відкрити файл для запису.
7. Записати новий розмір масиву та його елементи.

Алгоритм зчитування з файлу

1. Відкрити файл у режимі читання.
2. Зчитати `n`.
3. Зчитати `n` елементів масиву.
4. Закрити файл.

Алгоритм запису у файл

1. Відкрити файл у режимі запису.
2. Записати нову кількість елементів.
3. Записати всі елементи нового масиву.
4. Закрити файл.

Лістинг коду вирішення задач наведено в дод. А (стор.7-11).

Екран роботи програми показаний на рис. Б.1

Завдання 2.

Matrix23. Дана матриця розміру $M \times N$. У кожному рядку матриці знайти мінімальний елемент.

Вихідні дані (ім'я, опис, тип):

M — кількість рядків матриці, тип `int`, обмеження: $1 \leq M \leq 20$; N — кількість стовпців матриці, тип `int`, обмеження: $1 \leq N \leq 20$; `matr[M][N]` — цілочисельна матриця, тип `int`.

Вихідні дані (ім'я, опис, тип):

`min[M]` — масив, що містить мінімальний елемент кожного рядка матриці, тип `int`.

Алгоритм вирішення показано нижче

1. Відкрити файл.
2. Зчитати m і n .
3. Зчитати елементи матриці.
4. Для кожного рядка знайти мінімальний елемент.
5. Відкрити файл у режимі дописування.
6. Записати знайдені мінімуми у файл.
7. Закрити файл.

Алгоритм зчитування матриці

1. Відкрити файл у режимі читання.
2. Зчитати m і n .
3. Зчитати елементи матриці по рядках.
4. Закрити файл.

Алгоритм запису результатів

1. Відкрити файл у режимі дописування.
2. Обчислити мінімум кожного рядка.
3. Записати отримані мінімуми у файл.
4. Закрити файл.

Лістинг коду вирішення задачі наведено в дод. А (стор.7-11).

Екран роботи програми показаний на рис. Б.2

ВИСНОВКИ

У процесі виконання лабораторної роботи опановано методи опрацювання одновимірних і двовимірних масивів засобами мови C++. Закріплено практичні навички роботи з файлами, зокрема виконання операцій читання та запису даних. Реалізовано алгоритми обробки масивів і матриць, а також набуто досвіду структурування програмного коду.

ДОДАТОК А

Лістинг коду програми

```
#include "array_utils.h"
#include <fstream>
#include <iostream>

using namespace std;

// Read array from file
void get_mas(string filename, int arr[MAX], int &n)
{
    ifstream fin(filename);

    fin >> n;

    for(int i=0;i<n;i++)
        fin >> arr[i];

    fin.close();
}

// Write array to file
void show_mas(string filename, const int arr[MAX], const int n)
{
    ofstream fout(filename);

    fout << n << endl;

    for(int i=0;i<n;i++)
        fout << arr[i] << " ";

    fout.close();
}

// Array97 - keep only last occurrences
void removeDuplicatesKeepLast(int arr[MAX], int &n)
{
    for(int i=0;i<n;i++)
    {
        for(int j=i+1;j<n;j++)
        {
            if(arr[i]==arr[j])
            {
                for(int k=i;k<n-1;k++)
                    arr[k]=arr[k+1];

                n--;
                i--;
                break;
            }
        }
    }
}
```

```

        }
    }
}

// Bubble sort descending
void bubbleSortDesc(int arr[MAX], int n)
{
    for(int i=0;i<n-1;i++)
    {
        for(int j=0;j<n-i-1;j++)
        {
            if(arr[j] < arr[j+1])
            {
                int temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}

#include <iostream>
#include <string>
#include "array_utils.h"
#include "matrix_utils.h"

using namespace std;

// Task 1 - Array97
void task_array()
{
    string inFile, outFile;

    cout << "Enter input array file: ";
    cin >> inFile;

    cout << "Enter output file: ";
    cin >> outFile;

    int arr[MAX];
    int n;

    get_mas(inFile, arr, n);
    removeDuplicatesKeepLast(arr, n);
    show_mas(outFile, arr, n);

    cout << "Array processed and saved.\n";
}

// Task 2 - Matrix23

```



```

void task_matrix()
{
    string file;

    cout << "Enter matrix file: ";
    cin >> file;

    int matrix[MAXM][MAXM];
    int m,n;

    get_matrix(file, matrix, m, n);
    findRowMinimums(file, matrix, m, n);

    cout << "Row minimums added to file.\n";
}

// Task 3 - Bubble sort (descending)
void task_sort()
{
    string inFile,outFile;

    cout << "Enter array file: ";
    cin >> inFile;

    cout << "Enter output file: ";
    cin >> outFile;

    int arr[MAX];
    int n;

    get_mas(inFile, arr, n);
    bubbleSortDesc(arr, n);
    show_mas(outFile, arr, n);

    cout << "Array sorted.\n";
}

int main()
{
    int choice;

    do
    {
        cout << "\n===== MENU =====\n";
        cout << "1 - Array97\n";
        cout << "2 - Matrix23\n";
        cout << "3 - Bubble sort\n";
        cout << "0 - Exit\n";

        cin >> choice;
    }
}

```

```

        switch(choice)
        {
            case 1: task_array(); break;
            case 2: task_matrix(); break;
            case 3: task_sort(); break;
        }

    } while(choice != 0);

    return 0;
}
#include "matrix_utils.h"
#include <fstream>
#include <iostream>

using namespace std;

// Read matrix
void get_matrix(string filename, int matrix[MAXM][MAXM], int &m, int &n)
{
    ifstream fin(filename);

    fin >> m >> n;

    for(int i=0;i<m;i++)
        for(int j=0;j<n;j++)
            fin >> matrix[i][j];

    fin.close();
}

// Find minimum in each row and append to file
void findRowMinimums(string filename, int matrix[MAXM][MAXM], int m, int n)
{
    ofstream fout(filename, ios::app);

    fout << "\nMinimums in rows:\n";

    for(int i=0;i<m;i++)
    {
        int min = matrix[i][0];

        for(int j=1;j<n;j++)
            if(matrix[i][j] < min)
                min = matrix[i][j];

        fout << "Row " << i+1 << ": " << min << endl;
    }
}

```

```
        fout.close();
    }
#ifdef MATRIX_UTILS_H
#define MATRIX_UTILS_H

#include <string>
using namespace std;

const int MAXM = 20;

void get_matrix(string filename, int matrix[MAXM][MAXM], int &m, int &n);
void findRowMinimums(string filename, int matrix[MAXM][MAXM], int m, int n);

#endif
```

ДОДАТОК Б
Скрін-шоти вікна виконання програми

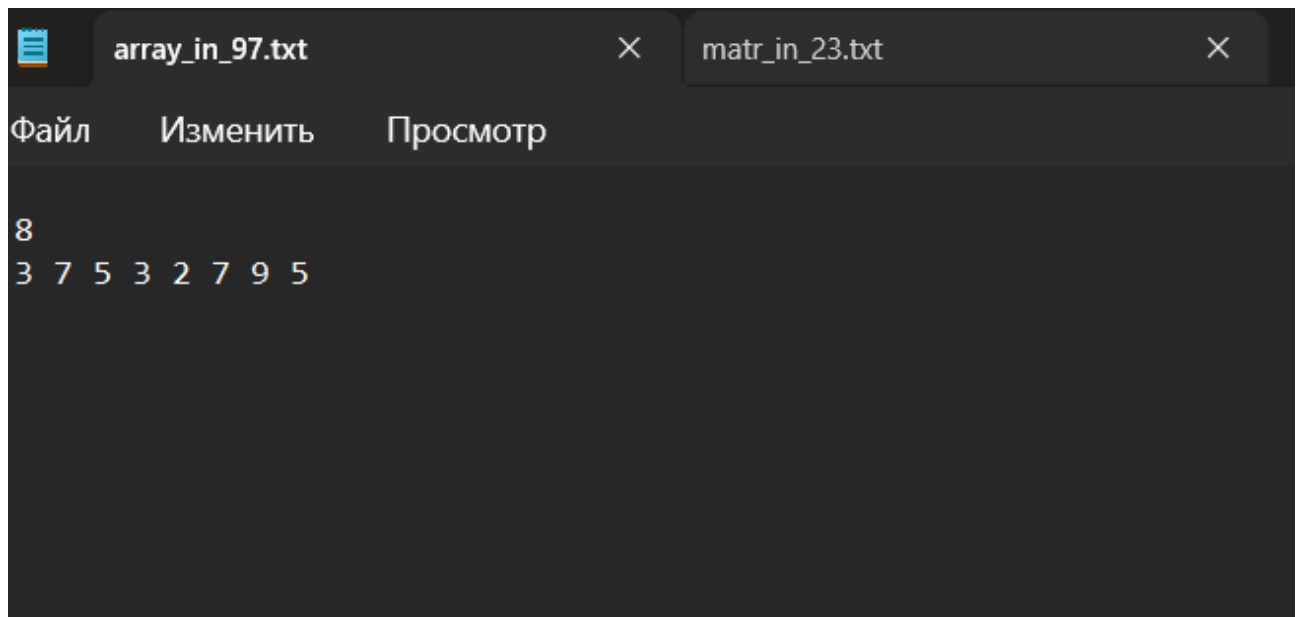


Рисунок Б.1 – Екран виконання програми для Завдання 1.Array97

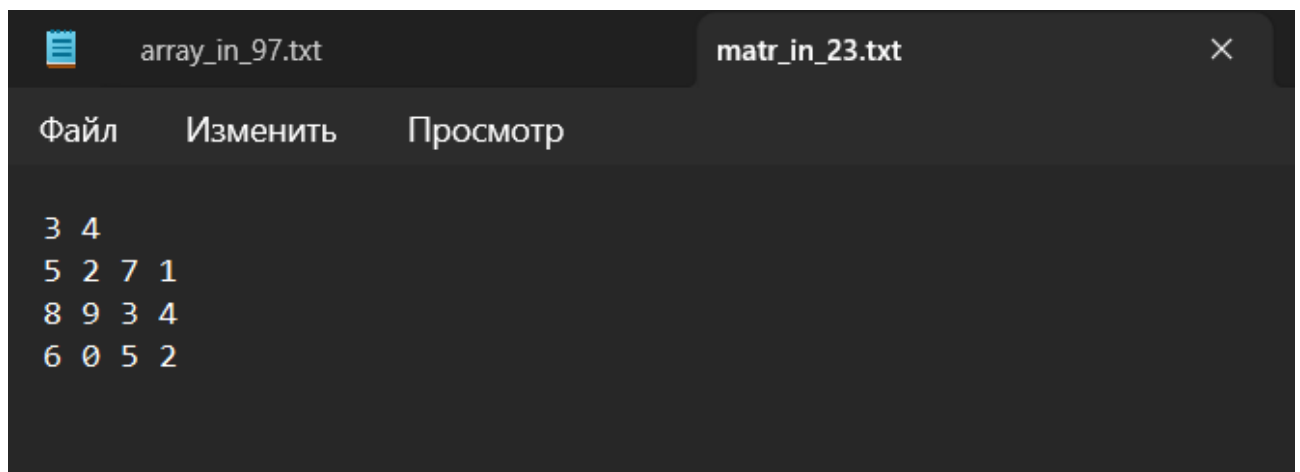


Рисунок Б.2 – Екран виконання програми для Завдання 2.Matrix23

ДОДАТОК В

Діалог з ШІ для самоаналізу

1. У функції `removeDuplicatesKeepLast` зі звіту використовується вкладений цикл зі зсувом елементів. Яка часова складність цього алгоритму в гіршому випадку?
 - A) $O(n)$
 - B) $O(n \log n)$
 - C) $O(n^2)$
 - D) $O(1)$
 2. Чому у функції `findRowMinimums` файл відкривається з прапорцем `ios::app`?
 - A) Щоб перезаписати файл
 - B) Щоб додати дані в кінець файлу
 - C) Щоб створити новий файл
 - D) Щоб прочитати файл
 3. У завданні `Array97` потрібно залишити лише останні входження елементів. Що станеться, якщо масив буде: `[2, 2, 2]`?
 - A) `[2, 2, 2]`
 - B) `[2]`
 - C) `[2, 2]`
 - D) `[]`
 4. Який режим відкриття файлу використовується в `show_mas`?
 - A) `ios::in`
 - B) `ios::app`
 - C) `ios::out` (без `app`)
 - D) `ios::binary`
 5. Чому в `bubbleSortDesc` масив сортується за спаданням, хоча завдання 3 вимагає «Зменшення»?
 - A) Помилка в коді
 - B) Це той самий алгоритм, що й за зростанням
 - C) «Зменшення» означає спадання
 - D) Сортування не впливає на результат
-

Відкриті питання

1. Поясніть, чому в алгоритмі `removeDuplicatesKeepLast` потрібно зменшувати індекс `i` та робити `break` після видалення елемента.
2. Як ви перевірили, що в матриці (завдання 2) правильно знаходяться мінімуми в кожному рядку, якщо у файлі є від'ємні числа?
3. Чи можна реалізувати завдання `Array97` без зсуву елементів, використовуючи допоміжний масив? Як це вплине на складність за пам'яттю?
4. У яких випадках бульбашкове сортування є ефективнішим, ніж, наприклад, швидке сортування?
5. Що станеться, якщо вхідний файл для завдання 1 буде порожнім або міститиме некоректні дані? Чи передбачено це у вашій програмі?

Користувач (студентка) надає відповіді (скорочений приклад):

1C, 2B, 3B, 4C, 5C.

Відкриті:

- 1 – Щоб повторно перевірити новий елемент на тій самій позиції.
- 2 – Перевірила вручну з тестовими даними, включаючи від'ємні.
- 3 – Можна, але буде $O(n)$ пам'яті, а час $O(n^2)$.
- 4 – Коли масив майже відсортований.
- 5 – Програма може впасти або видати помилку; перевірка не передбачена.

Користувач (студентка):

«Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку.»